Python programming

yen yung chin

February 23, 2020

Contents

1 變數、資料型態、輸出輸入 4
1.1 變數 4
1.1.1 命名規則 4
1.1.2 範例 4
1.1.3 變數的指派 (assign) 4
1.1.4 指派語法： 4
1.1.5 範例： 5
1.2 資料型態 5
1.2.1 常用類型 5
1.2.2 型態間的轉換 5
1.3 輸出: print 5
1.3.1 語法 5
1.3.2 範例 5
1.3.3 escape char 6
1.3.4 格式化輸出 6
1.3.5 format 格式化輸出 6
1.4 輸入 6
1.4.1 語法 6
1.4.2 範例 7
1.4.3 輸入搭配型別轉換 7
1.4.4 input 注意事項 7
1.5 註解 7
1.5.1 語法 7
1.5.2 範例 7
1.6 實作練習 8

2 運算元與運算式 8
2.1 算術運算 8
2.1.1 範例 #1 8
2.1.2 範例 #2 8
2.2 關係運算子 9
2.3 邏輯運算子 9
2.4 複合指定運算子 9

3 判斷結構 **9**
3.1 if . . . 9
3.1.1 語法 . . . 9
3.1.2 範例 . . . 9
3.2 if … else … . . . 9
3.2.1 語法 . . . 9
3.2.2 範例 . . . 10
3.3 if … elif … else … . . . 10
3.3.1 語法 . . . 10
3.3.2 範例 . . . 10
3.4 巢狀 if . . . 10
3.4.1 語法 . . . 10
3.4.2 範例: 某年份是否為閏年 . . . 11
3.4.3 code #1 . . . 11
3.4.4 code #2 . . . 11
3.4.5 練習題 . . . 11

4 迴圈結構 **11**
4.1 for . . . 11
4.1.1 語法 . . . 11
4.1.2 List 為 range() function 語法 . . . 12
4.1.3 List 為 range() function 語法 . . . 12
4.1.4 List 為 String . . . 12
4.2 while . . . 12
4.2.1 語法 . . . 12
4.2.2 範例 . . . 13
4.3 while + else . . . 13
4.3.1 語法 . . . 13
4.3.2 範例 . . . 13
4.4 break . . . 13
4.4.1 範例 . . . 13
4.5 continue . . . 14
4.5.1 範例 . . . 14
4.6 巢狀迴圈 . . . 14
4.6.1 範例 . . . 14

5 資料型別 **14**
5.1 字串 string . . . 14
5.1.1 資料格式 . . . 14
5.1.2 範例 . . . 14
5.1.3 字串運算子 . . . 15
5.1.4 字串 function 與 method . . . 15
5.2 列表 list . . . 16
5.2.1 資料格式 . . . 16
5.2.2 表列函式與方法 . . . 16
5.2.3 練習 . . . 17
5.3 元組 tuple . . . 18
5.3.1 資料格式 . . . 18
5.3.2 範例 . . . 18
5.3.3 練習: 史上最強掌法 . . . 18

5.3.4 解 1 . . . 19
5.3.5 解 2 . . . 19
5.4 字典 dict . . . 19
5.4.1 資料格式 . . . 19
5.4.2 範例 . . . 19
5.4.3 dic 操作 . . . 20
5.4.4 dic method . . . 20
5.5 List v.s. Tuple . . . 21
5.5.1 能否改變內容 . . . 21
5.5.2 可讀性 . . . 21
5.5.3 與 dictionary 的相似性 . . . 22
5.6 集合 set . . . 22
5.6.1 資料格式 . . . 22
5.6.2 範例 . . . 22
5.6.3 元素新增刪除 . . . 22
5.6.4 集合運算 . . . 23
5.6.5 詳細語法 . . . 23
5.6.6 練習：skyoj Contest Python 練習題 (資料型別) . . . 23
5.7 資料型別整理 . . . 23

6 FUNCTION **23**
6.1 函式 (function) 基本操作 . . . 23
6.1.1 函式 (function) 的重要性 . . . 23
6.1.2 自定義函式 . . . 24
6.1.3 自定義函式的呼叫 . . . 24
6.1.4 多參數函式 . . . 24
6.1.5 多回傳值函式 . . . 25
6.2 引數 (argument) 的傳遞 . . . 25
6.2.1 傳值與傳址 . . . 25
6.3 模組 (module) . . . 26
6.3.1 匯入模組語法： . . . 26
6.3.2 範例 . . . 26
6.4 遞迴 (recursive) . . . 26
6.4.1 語法 . . . 26
6.4.2 基本條件 . . . 27
6.4.3 範例 . . . 27
6.4.4 練習 . . . 27

7 Numpy 模組 **28**
7.1 匯入模組 . . . 28
7.2 Create ndarray . . . 28
7.2.1 一維陣列 . . . 28
7.2.2 多維陣列 . . . 28
7.3 矩陣運算 . . . 29
7.4 特殊矩陣 . . . 30
7.5 ndarray 屬性 . . . 31
7.5.1 練習 . . . 32
7.6 矩陣形狀轉換 . . . 32

8 Matplotlib **33**

9 網路爬蟲 **33**

10 Python 學習資源 **33**

11 0802 CNN **33**

12 以 python 下載網路資料 **33**

13 網路爬蟲 **33**
13.1 parse HTML 33
13.1.1 所需套件 33
13.2 範例: 以程式抓取 PTT 電影板 header 34
13.2.1 原則 34
13.2.2 實作: 取得 HTML 資料 34
13.2.3 實作：解析 HTML 資料內容 34
13.3 參考資料 36

14 如何把 python 程式 publish 成網站應用程式 **36**

1 變數、資料型態、輸出輸入

1.1 變數

1.1.1 命名規則

- 由英文、數字、底線、中文 (不建議) 組成
- 不得以數字開頭
- 不能與 Python 內建的保留字相同

1.1.2 範例

- abc_123:
- 3pigs
- Happy New Year
- Class
- Good!

1.1.3 變數的指派 (assign)

Python 變數不需宣告，依指派值自動設定資料型態。

1.1.4 指派語法：

- 變數名稱 = 指派值
- 變數不再使用時，可用 del 指令將其刪除，以節省記憶體。

1.1.5 範例:

```
a = 5
b = 3.14
c = 'TNFSH'
a = b = c = 10
name, number = 'TNFSH', 35
```

1.2 資料型態

1.2.1 常用類型

- 整數: int
- 浮點數: float
- 布林值 (True / False): bool, T 與 F 要大寫
- 字串: str, 以' 或" 含括, 若輸出字串要包含引號，則以另一種引號含括該字串。如:

```
good_words = " 請常說'請'、'謝謝'、'對不起' "
print(good_words)
```

請常說' 請'、' 謝謝'、' 對不起'

1.2.2 型態間的轉換

- 自動轉換

```
score = 60 + 3.5          # 自動轉換為浮點數，結果為63.5
```

- 強制轉換

```
score = int(30.22)            # 將括弧內的資料轉換為整數
score = float(score)          # 將括弧內的資料轉換為浮點數
test = str(score)                 # 將括弧內的資料轉換為字串
```

1.3 輸出: print

1.3.1 語法

- print(項目 1, [項目 2, ⋯, sep = 分隔字元, end = 結束字元])
- sep (分隔字元) 預設為空白字元
- end (結束字元) 預設為換行字元

1.3.2 範例

```
a, b = 5, 10
print(a, b)
print(a, b, sep =',')
print(a, end='\t')
print(b)
print(a, end='')
print(b)
```

```
5 10
5,10
5        10
510
```

1.3.3 escape char

跳脫字元	說明
\	印出 \
\'	印出'
	換行字元
^	水平跳格字元 (Tab)
_	倒退一格字元 (Backspace)

1.3.4 格式化輸出

1. 語法
   - 格式化輸出語法：print(字串%(參數))
   - 字串裡%s 代表字串、%d 代表整數、%f 代表浮點數
2. 範例

```
name = '台北101'
height = 508
fee = 18.24
print('%s的高度為%d公尺，參觀門票為%3.2f美金.' %(name, height, fee))
```

台北 101 的高度為 508 公尺，參觀門票為 18.24 美金.

1.3.5 format 格式化輸出

1. 語法
   - format 格式化輸出語法：print(字串.format (參數))
   - 字串裡以 {0}、{1}、... 來對應參數列裡的變數
2. 範例

```
name = 'Taipei101'
height = 508.35
print('{0}的高度為{1}公尺'.format (name, height))
print('{0:15s}的高度為{1:10.1f}公尺'.format (name, height))
```

```
Taipei101 的高度為 508.35 公尺
Taipei101       的高度為      508.4 公尺
```

1.4 輸入

1.4.1 語法

```
variable = intpu([提示字元])
```

- PS: 經由 input( ) 函式讀入的資料，其資料型態皆為字串

1.4.2 範例

```
a = input('輸入國文成績:')
b = input('輸入數學成績:')
c = input()
print('三科成績分別為%5s %5s %5s' %(a, b, c))
```

```
name = input()
age = input()
print("My name is " + name + ", and I am " + age + " years old.")
```

1.4.3 輸入搭配型別轉換

1. 範例

```
a = int(input('輸入國文成績:'))
b = int(input('輸入數學成績:'))
c = int(input())
print('總分為%5d' %(a+b+c))
```

1.4.4 input 注意事項

- input 每筆輸入讀至換行符號為止
- 若測試資料輸入以空白間隔，則讀入整行字串後， 再以字串分割處理
- Inline code `echo -e "test"`

```
inp = input()
inpstr = inp.split()
# 依此類推逐一取得輸入值
a = int(inpstr[0])
b = int(inpstr[1])
c = int(inpstr[2])
```

1.5 註解

1.5.1 語法

- 單行註解：以 # 開頭
- 多行註解：前後以"' 或""" 含括

1.5.2 範例

```
## 這是單行註解
int a = 3
'''
這是多行註解
LALALA
'''
print(a)
"""
這也是
"""
```

1.6 實作練習

- skyoj #161 華氏溫度轉攝氏溫度

2 運算元與運算式

2.1 算術運算

- +: 加
- -: 減
- *: 乘
- /: 除
- %: 取餘數
- //: 求商
- **: 指數

2.1.1 範例 #1

```python
# python code for arithematic opearation
print(5+3)
print(5-3)
print(5*3)
print(5/3)
print(5%3)
print(5//3)
print(5**3)
```

```
8
2
15
1.6666666666666667
2
1
125
```

2.1.2 範例 #2

```python
a = int(input("input a: "))
op = input("input op: ")
b = int(input("input b: "))

if op == '+':
    ans = a + b;
elif op == '-':
    ans = a - b;
elif op == '*':
    ans = a * b;
elif op == '/':
```

```
    ans = a / b;
elif op == '%':
    ans = a % b;

print(str(a) + op + str(b) + "=" + str(ans))
```

2.2 關係運算子

>	<	>=	<=	==	!=
大於	小於	大於等於	小於等於	等於	不等於

2.3 邏輯運算子

and	or	not
(a>b)and(a<c)	(a>b)or<(a==b)	not(a>b)

2.4 複合指定運算子

+=	-=	*=	/=	%=	//=	**=

3 判斷結構

3.1 if

3.1.1 語法

- 條件式可不用括號 ( ) 含括，條件式後需搭配冒號：
- 程式區塊以縮排方式處理，同一層縮排視為同一程式區塊

```
if condition:
    statement 1
    ...
```

3.1.2 範例

```
num = 31
if num % 2 == 0:
    print('%d is even' %(num))
if num % 2 == 1:
    print('%d is odd' %(num))
```

```
31 is odd
```

3.2 if ... else ...

3.2.1 語法

```
if condition:
    statement 1
    ...
else:
    statement 3
    ...
```

3.2.2 範例

```
num = 32
if num % 2 == 0:
    print('%d is even' %(num))
else:
    print('%d is odd' %(num))
```

3.3 if ... elif ... else ...

3.3.1 語法

```
if condition 1:
    statement 1
    ...
elif condition 2:
    statement 3
    ...
elif condition 3:
    statement 5
    ...
else:
    statement N
```

3.3.2 範例

```
score = 87
if score >= 90:
    print('A')
elif score >= 80:
    print('B')
elif score >= 70:
    print('C')
elif score >= 60:
    print('D')
else:
    print('F')
```

3.4 巢狀 if

3.4.1 語法

```
if condition 1:
    statement 1
    ...
    if condition 2:
        statement 3
        ...
    else:
        statement 5
        ...
else:
    if condition 3:
        statement 7
        ...
    else:
        statement N
        ...
```

3.4.2 範例: 某年份是否為閏年

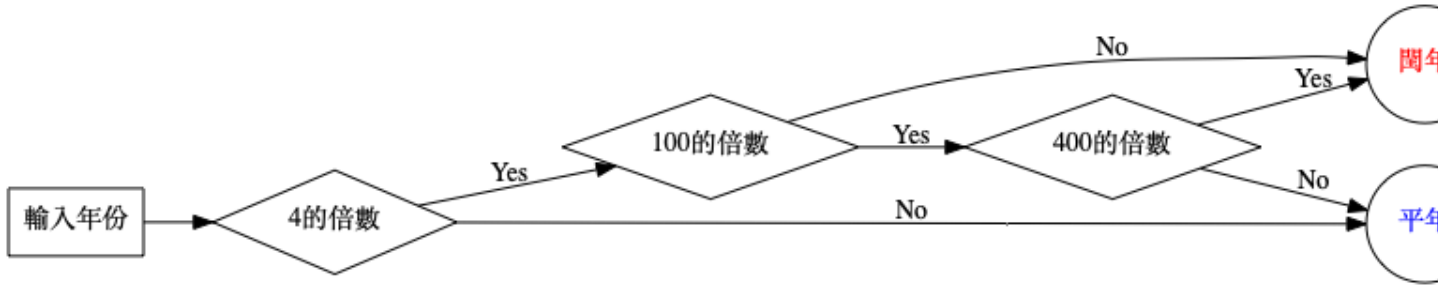

Figure 1: 閏年判斷流程

3.4.3 code #1

```
year = int(input("請輸入一個年份:"))
if (year % 4) == 0:
    if (year % 100) == 0:
        if (year % 400) == 0:
            print("%s年是世紀閏年" % year)
        else:
            print("%s年為平年" % year)
    else:
        print("%s年是普通閏年" % year)
else:
    print("%s年為平年" % year)
```

3.4.4 code #2

```
year = int(input("請輸入一個年份:"))
if (year % 4) == 0 and (year % 100) !=0 or (year % 400) == 0:
    print("%s年是閏年" % year)
else:
    print("%s年為平年" % year)
```

3.4.5 練習題

- skyoj #162 成績等第判別

4 迴圈結構

4.1 for

4.1.1 語法

```
for variable in List:
    statement 1
    ...
```

- for 迴圈的變數會依序走訪序列中的元素
- 序列[1]可為 range 函式、字串 (string)、表列 (list)、元組 (tuple)、字典 (dict)、集合 (set)

[1]Python 數據類型-List 介紹

4.1.2 List 為 range() function 語法

```
for variable in range([起始值,] 終止值 [,遞增值]):
    statement 1
    ...
```

- 起始值預設為 0，遞增值預設為 1
- 起始值　range( ) 的範圍 < 終止值

4.1.3 List 為 range() function 語法

```
for x in range(6):
    print(x, end=' ')
print('\n====')
for x in range(3, 10, 2):
    print(x, ' ')
print('\n====')
for x in range(3, 10):
    print(x, end=' ')
```

```
0 1 2 3 4 5
====
3
5
7
9

====
3 4 5 6 7 8 9
```

4.1.4 List 為 String

```
school = 'TNFSH'
for x in school:
    print(x, end=' ')
print('\n=========')
for x in school:
    print(chr(ord(x)+1), end=' ')
```

```
T N F S H
=========
U O G T I
```

- ord( ) -> 將字元轉為 ASCII 編碼 (整數)
- chr( ) -> 將 ASCII 編碼 (整數) 轉為字元

4.2 while

4.2.1 語法

```
while (condition):
    statement 1
    ...
```

4.2.2 範例

```
n = 12345
while (n > 0):
    print(n%10)
    n //= 10
```

```
5
4
3
2
1
```

```
m, n = 42, 75
while (n > 0):
    m, n = n, m % n
print(m)
```

```
3
```

4.3 while + else

4.3.1 語法

```
while (condition):
    statement 1
    ...
else:
    statement 3
    ...
```

4.3.2 範例

```
n = 56
while (n%2 == 0):
    print(n, end=' ')
    n //= 2
else:
    print('end with %d' %n)
```

```
56 28 14 end with 7
```

4.4 break

跳出整個 loop

4.4.1 範例

```
for letter in 'Python':
    if letter == 'h':
        print('Now stop...')
        break
    print('Processing Letrer:', letter)
```

```
Processing Letrer: P
Processing Letrer: y
Processing Letrer: t
Now stop...
```

4.5 continue

跳出單次 loop

4.5.1 範例

```
for letter in 'Python':
    if letter == 'h':
        print('Now skip...')
        continue
    print('Processing Letrer:', letter)
```

```
Processing Letrer: P
Processing Letrer: y
Processing Letrer: t
Now skip...
Processing Letrer: o
Processing Letrer: n
```

4.6 巢狀迴圈

4.6.1 範例

```
for i in range(1, 10):
    for j in range(1, 10):
        print('%d*%d=%2d' %(i, j, i*j), end=' ')
    print()
```

```
1*1= 1 1*2= 2 1*3= 3 1*4= 4 1*5= 5 1*6= 6 1*7= 7 1*8= 8 1*9= 9
2*1= 2 2*2= 4 2*3= 6 2*4= 8 2*5=10 2*6=12 2*7=14 2*8=16 2*9=18
3*1= 3 3*2= 6 3*3= 9 3*4=12 3*5=15 3*6=18 3*7=21 3*8=24 3*9=27
4*1= 4 4*2= 8 4*3=12 4*4=16 4*5=20 4*6=24 4*7=28 4*8=32 4*9=36
5*1= 5 5*2=10 5*3=15 5*4=20 5*5=25 5*6=30 5*7=35 5*8=40 5*9=45
6*1= 6 6*2=12 6*3=18 6*4=24 6*5=30 6*6=36 6*7=42 6*8=48 6*9=54
7*1= 7 7*2=14 7*3=21 7*4=28 7*5=35 7*6=42 7*7=49 7*8=56 7*9=63
8*1= 8 8*2=16 8*3=24 8*4=32 8*5=40 8*6=48 8*7=56 8*8=64 8*9=72
9*1= 9 9*2=18 9*3=27 9*4=36 9*5=45 9*6=54 9*7=63 9*8=72 9*9=81
```

5 資料型別

5.1 字串 string

5.1.1 資料格式

以單引號 (') 或雙引號 (") 將資料含括起來

5.1.2 範例

index 左至右	0	1	2	3	4	5	6	7	8	9
字串	0	6	-	2	3	7	1	2	0	6
index 右至左	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

5.1.3 字串運算子

- +: 連接字串
- *: 重複字串
- [index]: 字串中 index 所在字元
- [start:end:increment]: 截取部份字串
- in: 判斷是否為子字串

```
tel = '06-2371206'
ext = '#600'
# +
print('tel+ext:',tel+ext)
# *
print('ext*2:', ext*2)
# [index]
print('tel[5]:', tel[5])
# [start:end:increment]
print('tel[1:4]:',tel[1:4])
print('tel[6: ]:',tel[6: ])
print('tel[ :6]:',tel[ :6])
print('tel[::-1]:',tel[::-1])
# in
print("'9' in tel:", '9' in tel)
```

```
tel+ext: 06-2371206#600
ext*2: #600#600
tel[5]: 7
tel[1:4]: 6-2
tel[6: ]: 1206
tel[ :6]: 06-237
tel[::-1]: 6021732-60
'9' in tel: False
```

5.1.4 字串 function 與 method

- len(<str>): 計算字串長度
- <str>.lower(): 字串轉小寫
- <str>.upper(): 字串轉大寫
- <str>.islower(): 字串中英文全大寫
- <str>.isupper(): 字串中英文全小寫
- <str>.find(<str1>): 在 <str> 尋找 <str1>，回傳索引值； 若未找到，回傳-1
- <str>.replace(<str1>, <str2>): 將 <str> 中的 <str1> 以 <str2> 取代
- <str>.split([sep]): 字串以 sep 分割, sep 預設值為空白

```python
school = 'Tnfsh'
print('school:', school)
print('len(school):', len(school))
print('school.lower():', school.lower())
print('school.isupper():', school.isupper())
print("school.find('fsh'):", school.find('fsh'))
print("school.replace('fsh', ssh'):", school.replace('fsh', 'ssh'))
school = school.replace('fsh', 'ssh')
print("school.split('f'):", school.split('n'))
```

```
school: Tnfsh
len(school): 5
school.lower(): tnfsh
school.isupper(): False
school.find('fsh'): 2
school.replace('fsh', ssh'): Tnssh
school.split('f'): ['T', 'ssh']
```

5.2 列表 list

5.2.1 資料格式

- 以 [ ] 將不同型態的資料含括起來，以, 分隔
- 表列中的資料是有序排列，從 0 開始編號
- 格式：表列名稱 = [元素 1, 元素 2, ⋯]

```python
data = ['John', [95, 118], 'May', 100]
print(data[1])
```

```
[95, 118]
```

5.2.2 表列函式與方法

- len(<list>): 計算表列元素個數
- list(<str>): 將 <str> 轉成表列
- <list>.clear(): 清除表列中所有元素
- <list>.append(<obj>): 將 <obj> 加到 <list> 尾端
- <list>.extend(<list1>): 將 <list1> 合併至 <list> 尾端
- <list>.remove(<obj>): 移除 <list> 中 <obj> 元素
- <list>.insert(<i>,<obj>): 將 <obj> 加到 <list> 的索引值 <i> 位置
- <list>.pop([index]): 從 <list> 取出指定元素　若 [index] 未指定，則取出尾端元素
- <list>.reverse(): 反轉表列元素

- sum(<list>): 將表列中所有元素加總 (註：僅限表列元素皆為數字)
- <list>.sort(): 將表列中所有元素排序
- '[sep]'.join(<list>): 用 [sep] 來連結表列元素，若未指定 [sep]，則無間隔 (僅限表列元素皆為字串)

```
data = ['John', [95, 118], 'May', 100]
print(len(data))
print(len(data[1]))
print(list('TNFSH'))
data.clear()
print(data)
data.append(35)
print(data)
data1 = [89, 'James', 100]
data.extend(data1)
print(data)
data.remove('James')
print(data)
data.insert(1, 200)
print(data)
data.pop()
data.reverse()
print(data)
print(sum(data))
data.sort()
print(data)
# string
data_s = ["a", "b", "c"]
sep = "-"
print(sep.join(data_s))
```

```
4
2
['T', 'N', 'F', 'S', 'H']
[]
[35]
[35, 89, 'James', 100]
[35, 89, 100]
[35, 200, 89, 100]
[89, 200, 35]
324
[35, 89, 200]
a-b-c
```

5.2.3 練習

- 連續輸入成績 (輸入-1 結束)，將成績由高至低排序輸出，並輸出總分、平均

```
score = []
while(True):
    sc = int(input())
    if (sc != -1):
        score.append(sc)
    else:
```

```
        break
score.sort(reverse=True)
print(score)
```

- 計算機

```
sum = 0
grades = []
for i in range(0, 5):
    x = int(input("請輸入第" + str(i + 1) + "個成績: "))
    grades.append(x)
    sum += x
print(sum/5)
print(grades)
```

5.3 元組 tuple

5.3.1 資料格式

- 資料格式：以 ( ) 將不同型態的資料含括起來，以, 分隔
- 元組中的資料是有序排列，從 0 開始編號
- 格式：元組名稱 = (元素 1, 元素 2, ⋯)
- 與表列 list 類似，但元組 tuple 內的元素不能修改

5.3.2 範例

```
data = ( 'John', [95, 118], 'May', 100 )
print(data[1])
tup1 = (53, )    #只有一個元素時，其後要加逗號
print(tup1)
```

```
[95, 118]
(53,)
```

5.3.3 練習：史上最強掌法

自從鐵掌無敵馬掌門出任武林盟主後，綠林豪傑一片哀鴻，民不聊生，痛苦指數破表，眾家名門弟子甚至遠避異鄉以求溫飽。近來馬掌門更獨創「油電雙掌」，以圖鞏固武林領導地位，此套掌法高深莫測，武林中人聞風喪膽，少林、武當各派高手不得不拋開門戶之見，齊聚「竹園崗」合作商議對策。目前僅由歷來幾次掌下逃生者的對戰經驗分析出以下數據

與馬掌門對戰時間 (秒)	對戰者每秒接受之傷害值
120 以下	2.10
121~330	3.02
331~500	4.39
501~700	4.97
701 以上	5.63

請你幫這些可憐的高手寫一個程式分析對戰時間與受傷指數間的關係。
輸入：0 ~ 10000 秒之間任意值
輸出：受傷指數

5.3.4 解 1

```
#sec = input()
sec = 800
if sec <= 120:
    print(sec*2.10)
elif sec > 120 and sec <= 330:
    print(120*2.10+(sec-120)*3.02)
elif sec > 330 and sec <= 500:
    print(120*2.10+(330-120)*3.02+(sec-300)*4.39)
elif sec > 500 and sec <= 700:
    print(120*2.10+(330-120)*3.02+(500-330)*4.39+(sec-500)*4.97)
elif sec > 700:
    print(120*2.10+(330-120)*3.02+(500-330)*4.39+(700-500)*4.97+(sec-700)*5.63)
```

```
3189.5
```

5.3.5 解 2

```
#sec = input()
sec = 800
hurt = 0
gap = (700, 500, 330, 120, 0)
rate = (5.63, 4.97, 4.39, 3.02, 2.10)
for i in range(5):
    if sec > gap[i]:
        hurt += (sec-gap[i])*rate[i]
        sec = gap[i]
print(hurt)
```

```
3189.5
```

5.4 字典 dict

5.4.1 資料格式

- 以 { } 將各組鍵: 值對應資料含括起來，以, 分隔
- 字典中的資料是無序的
- 格式：字典名稱 = {k1:v1, k2:v2, ⋯}
- 範例：
- 註：若字典中有相同的 key，則會取出最後的 value

5.4.2 範例

```
data = { 'John': 95,  'May': 100 }
print(data['May'])
```

```
100
```

5.4.3 dic 操作

- 新增、修改、刪除

```
data = { 'John': 95,  'May': 100 }
# 新增
data['Harrison'] = 88
# 修改
data['John'] = 99
# 刪除鍵值對
del data['John']
print(data)
# 刪除dic
del data
# print(data) --> NameError: name 'data' is not defined
```

```
{'May': 100, 'Harrison': 88}
```

5.4.4 dic method

- <dict>.clear()

```
data = { 'John': 95,  'May': 100, 'John': 105 }
print("data:",data)
print("data.clear():",data.clear())
```

```
data: {'John': 105, 'May': 100}
data.clear(): None
```

- <dict>.copy()

```
data = { 'John': 95,  'May': 100, 'John': 105 }
data1 = data.copy()
print("data1 = data.copy():",data1.copy())
```

```
data1 = data.copy(): {'John': 105, 'May': 100}
```

- <dict>.get(key, default=None)
- <dict>.items()
- <dict>.keys()
- <dict>.values()

```
data = { 'John': 95,  'May': 100, 'John': 105 }
print("data.get('John'):",data.get('John'))
print("data.items():",data.items())
print("data.keys():",data.keys())
print("data.values()",data.values())
```

```
data.get('John'): 105
data.items(): dict_items([('John', 105), ('May', 100)])
data.keys(): dict_keys(['John', 'May'])
data.values() dict_values([105, 100])
```

- <dict>.popitem()

```
data = { 'John': 95,  'May': 100, 'John': 105 }
pop1 = data.popitem()
print(pop1)
```

```
('May', 100)
```

- <dict>.fromkes(<seq>[, val]): Python 字典 fromkeys() 函數用於創建一個新字典，以序列 seq 中元素做字典的鍵，value 為字典所有鍵對應的初始值。

```
data = { 'John': 95,  'May': 100, 'John': 105 }
name = ('Vanissa', 'May', 'John')
data2 = data.fromkeys(name)
print(data2)
data2 = data.fromkeys(name, 100)
print(data2)
```

```
{'Vanissa': None, 'May': None, 'John': None}
{'Vanissa': 100, 'May': 100, 'John': 100}
```

5.5 List v.s. Tuple

二者的差異與適用時機[2]：

5.5.1 能否改變內容

Using a tuple instead of a list can give the programmer and the interpreter a hint that the data should not be changed.

5.5.2 可讀性

Reading data is simpler when tuples are stored inside a list. For example,

```
[(2,4), (5,7), (3,8), (5,9)]
```

is easier to read than

```
[[2,4], [5,7], [3,8], [5,9]]
```

[2]Python List vs. Tuples

5.5.3 與 dictionary 的相似性

Tuples are commonly used as the equivalent of a dictionary without keys to store data. For Example,

```
[('Swordfish', 'Dominic Sena', 2001), ('Snowden', ' Oliver Stone', 2016), ('Taxi
    Driver', 'Martin Scorsese', 1976)]
```

Above example contains tuples inside list which has a list of movies.
Tuple can also be used as key in dictionary due to their hashable and immutable nature whereas Lists are not used as key in a dictionary because list can't handle $__hash__()$ and have mutable nature.

```
  key_val= {('alpha','bravo'):123} #Valid
key_val = {['alpha','bravo']:123} #Invalid
```

5.6 集合 set

5.6.1 資料格式

- 資料格式：以 {} 將各組資料含括起來，以',' 分隔, 或以 set() 建立
- 集合中的資料是無序的，會自動刪除重複元素
- 格式： `集合名稱 = {元素1, 元素2, ⋯ }`
- 註：set(<seq>) 函式的參數 <seq> 可為字串、表列、元組、字典

5.6.2 範例

```
S1 = { 'John', 95, 'May', 100, 'John' }
S2 = set('apple')    {'l', 'a', 'e', 'p'}  # 集合的資料是無序的
```

5.6.3 元素新增刪除

- `<set>.add(<item>)`
- `<set>.remove(<item>)`
- remove() 若無此 item 會發生錯誤

```
s = set('apple')
s.add('x')
print(s)
s.remove('p')
print(s)
```

```
{'e', 'p', 'x', 'l', 'a'}
{'e', 'x', 'l', 'a'}
```

5.6.4 集合運算

運算	符號
聯集	|
交集	&
差集	-
互斥	^
屬於	in

```
c = {'1234', 'TNFSH', 'qq', '434'}
print(c)
a = set('1234')
b = set('357')
print("a | b:", a | b)
print("a & b:", a & b)
print("a - b:", a - b)
print("b | a:", b | a)
print("a ^ b:", a ^ b)
print("'5' in a:", '5' in a)
print("'5' in b:", '5' in b)
```

```
{'1234', 'TNFSH', '434', 'qq'}
a | b: {'4', '2', '1', '5', '7', '3'}
a & b: {'3'}
a - b: {'4', '2', '1'}
b | a: {'4', '5', '2', '1', '7', '3'}
a ^ b: {'4', '2', '7', '5', '1'}
'5' in a: False
'5' in b: True
```

5.6.5 詳細語法

- Python3 集合

5.6.6 練習：skyoj Contest Python 練習題 (資料型別)

5.7 資料型別整理

資料型別	符號	資料可改?	資料有序?	資料類型
字串 string	' ' 或" "			相同 (字元)
表列 list	[ ]			可不同
元組 tuple	( )			可不同
字典 dict	{ }			相同 (鍵: 值)
集合 set	{ }			可不同

6 FUNCTION

6.1 函式 (function) 基本操作

6.1.1 函式 (function) 的重要性

- function 為一段程式的集合，可視為一個獨立的區段。

- function 可重覆使用，因此為結構化程式語言的重要元素。 可將龐大複雜的程式分解為小問題，由多人分別以 function 解決，縮短開發時間。

- function 可區分為以下三類：
  - Python 內建 function，如: print()、len()、int()、str()…。
  - 第三方公司所開發的模組 module(多個 function 組合)。
  - 自定義 function。

6.1.2 自定義函式

- 宣告語法

```
def 函式名稱 ( [參數1, 參數2, ...] ):
    程式區塊
    [ return 值 ]
```

- 說明:
  - 參數可省略，亦即呼叫 function 不需傳入任何資料 (引數)。
  - 若無回傳值，則無需 return。
  - 若多個回傳值可用逗號隔開。

6.1.3 自定義函式的呼叫

- 呼叫函式語法：

```
def circle(r):
    return r*r*3.14;

r = 10
print('半徑%d的圓面積: %d' %(r, circle(r)))
```

```
半徑 10 的圓面積: 314
```

6.1.4 多參數函式

- 呼叫函式時引數 (arugment) 個數
- 自定義函式的參數 (parameter) 個數

設定參數初始值。 若呼叫時無引數傳入， 則使用其初始值。

```
def printInfo(name, school='TNFSH'):
    print('學生:',name,'就讀:',school)
    return

printInfo(name='James')
printInfo(name='Harrison',school='TNSSH')
```

```
學生: James 就讀: TNFSH
學生: Harrison 就讀: TNSSH
```

6.1.5 多回傳值函式

- 呼叫函式多個回傳值語法：

  –

```
def scoreProcessor(score):
    sum = 0
    for x in score:
        sum += x
    return sum, sum/len(score)

scores = (21, 33, 83, 100, 75, 60)
sum, avg = scoreProcessor(scores)
print('sum=',sum,',average=',avg)
```

```
sum= 372 ,average= 62.0
```

6.2 引數 (argument) 的傳遞

- 每一個函式都是獨立的，所以函式只知道自己程式區域內的變數，不認識函式外的變數。
- 變數因其所宣告位置，可區分為全域變數 (Global variable)、區域變數 (Local variable)。
  - 全域變數：宣告在任何函式外，所有函式皆可存取。
  - 區域變數：宣告在函式內，僅函式內可存取。c
- 全域變數與區域變數範例

```
def printGlobal(x, y):
    print(a+b)

def printLocal(a, b):
    print(a+b)

a, b = 5, 10
printGlobal(10, 10)
printLocal(10,10)
```

```
15
20
```

6.2.1 傳值與傳址

- 引數 (argument) 的傳遞分為兩種方式，由 Python 自動判別：
  - 傳值呼叫 (Call-by-Value)
  - 傳址呼叫 (Call-by-Reference)

引數傳遞	傳遞內容	參數改變	資料型別
傳值呼叫 call-by-value	引數值	否	bool, int, float, str, tuple, chr
傳址呼叫 call-by-reference	引數記憶體位址	是	list, dict, set

```python
# str傳值、list傳址
def ccc(name, score):
    name = 'Amy'
    score.append(99)
    print('--函式內輸出---')
    print('name:', name,'\tscore:',score)

name = 'Josh'
score = [77, 88]

ccc(name, score)
print('--主程式輸出--')
print('name:', name,'\tscore:',score)
```

```
--函式內輸出---
name: Amy         score: [77, 88, 99]
--主程式輸出--
name: Josh         score: [77, 88, 99]
```

6.3 模組 (module)

6.3.1 匯入模組語法：

-
-

6.3.2 範例

```python
import random as ran

print(ran.randint(1, 100))
print(ran.randint(1, 100))
print(ran.randint(1, 100))
```

```
50
78
16
```

6.4 遞迴 (recursive)

6.4.1 語法

- 遞迴 (recursive)：一個函式直接呼叫函式本身

```python
def fib(n):
    if n == 1 or n == 2:
        return 1
    else:
        return fib(n-1) + fib(n-2)

print('The %d-th Fibonacci number = %d' %(10, fib(10)))
```

```
The 10-th Fibonacci number = 55
```

6.4.2 基本條件

- 遞迴程式設計要滿足兩條件:

  1. 遞迴關係式，如

```
fib(n) = fib(n-1) + fib(n-2)
sum(n) = n + sum(n-1)
```

  1. 遞迴終止式，如

```
f(1) = f(2) = 1
sum(0) = 0
```

- 範例:GCD-while

```
m, n = 20, 50
while (n>0):
    m, n = n, m % n
print(m)
```

```
10
```

6.4.3 範例

- 範例:GCD-recursive

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

a, b = 10, 50
print(gcd(a, b))
```

```
10
```

6.4.4 練習

- 計算 n! 值。 輸入 n，輸出 n! = XXXXX。如輸入 5，輸出 5! = 120。
- 計算 C(n, r) 組合數。 輸入 n, r，輸出 C(n, r) = XX。如輸入 9, 4，輸出 C(9, 4) = 126。
- 計算擴展歐幾里德。 輸入 a, b，輸出 a*x + b*y = gcd。如輸入 3, 5，輸出 3*2 + 5*-1 = 1

```
def ext_euclid(a, b):
    old_s,s=1,0
    old_t,t=0,1
    old_r,r=a,b
    if b == 0:
        return 1, 0, a
    else:
        while(r!=0):
            q=old_r//r
```

```
            old_r,r=r,old_r-q*r
            old_s,s=s,old_s-q*s
            old_t,t=t,old_t-q*t
    return old_s, old_t, old_r

a, b, c = ext_euclid(240, 33)
print('%dx + %dy = %d' %(a, b, c))
```

```
4x + -29y = 3
```

7 Numpy 模組

7.1 匯入模組

- 使用模組裡的函式要加模組名稱，如 numpy.arrry( )。例： `import numpy`
- 匯入 numpy 模組並使用 np 作為簡寫，這是 Numpy 官方倡導的寫法 `import numpy as np`
- Numpy 中的多維資料型別稱為 ndarray

7.2 Create ndarray

7.2.1 一維陣列

- list 或 tuple 轉入

```
import numpy as np
np1 = np.array( [1, 2, 3, 4] )
print(np1)
```

```
[1 2 3 4]
```

- 使用 np.arange( ) 方法

```
import numpy as np
np2 = np.arange(5)
print(np2)
```

```
[0 1 2 3 4]
```

7.2.2 多維陣列

- 二維陣列

```
import numpy as np
np4 = np.array( [[1, 2, 4], [3,4,5]])
print('np4:', np4)

np5 = np.array([np.arange(3), np.arange(3)])
print('np5:', np5)

np6 = np.arange(8).reshape(2, 4)
print('np6:', np6)
```

```
np4: [[1 2 4]
 [3 4 5]]
np5: [[0 1 2]
 [0 1 2]]
np6: [[0 1 2 3]
 [4 5 6 7]]
```

- 多維陣列

```python
import numpy as np

np7 = np.arange(24).reshape(2, 3, 4)
print('np7:',np7)
```

```
np7: [[[ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]
```

- 隨機矩陣

```python
import numpy as np

np8 = np.random.random( (3, 2)) #矩陣大小以tuple表示
print('np8:', np8)

np9 = np.random.randint( 0, 100, size=[4, 3]) #矩陣大小以list表示
print('np9:', np9)
```

```
np8: [[0.04349769 0.48755705]
 [0.50627204 0.56730179]
 [0.56445113 0.31159334]]
np9: [[53 73 52]
 [60 58 33]
 [69 89 98]
 [17 92 90]]
```

7.3 矩陣運算

```python
import numpy as np

A = np.array([
    [5, 3, 7],
    [0, 1, -6]
])

B = np.array([
    [1, 2, 3],
    [-1, -2, -3]
])

```

```
C = np.array([
    [1, 2],
    [-1, -2],
    [0, 5]
])

k = -3

print('---A+B---\n', A+B)
print('---A-B---\n', A-B)
print('---k*B---\n', k*B)
print('---A.dot(C)---\n', A.dot(C))
```

```
---A+B---
 [[ 6  5 10]
 [-1 -1 -9]]
---A-B---
 [[ 4  1  4]
 [ 1  3 -3]]
---k*B---
 [[-3 -6 -9]
 [ 3  6  9]]
---A.dot(C)---
 [[  2  39]
 [ -1 -32]]
```

7.4 特殊矩陣

- 元素全為 0 矩陣： `np.zeros(<大小>, <資料型別>)`
- 元素全為 1 矩陣： `np.ones(<大小>, <資料型別>) # 矩陣大小以 tuple 指定`
- 單位矩陣 (Identity)： `np.eye(<m>, <資料型別>)# eye 與 I 同音`
- 反矩陣 (Inverse)： `np.linalg.inv(<矩陣>) # 行列式不可為0`

```
import numpy as np

print('np.zeros:\n', np.zeros((3,5), dtype='int'))
print('np.ones:\n', np.ones((3,5), dtype='int'))
print('np.eyes:\n', np.eye(4, dtype='int'))
x = np.random.randint( 0, 10, size=[3, 3])
print('x:\n', x)
xt = x.T # 轉置
print('xt:\n', xt)
mat = xt.dot(x)
print('mat:\n', mat)
print('np.linalg.inv(mat):\n', np.linalg.inv(mat))
```

```
np.zeros:
 [[0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]]
np.ones:
```

```
 [[1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]]
np.eyes:
 [[1 0 0 0]
 [0 1 0 0]
 [0 0 1 0]
 [0 0 0 1]]
x:
 [[5 7 1]
 [2 8 6]
 [1 6 3]]
xt:
 [[5 2 1]
 [7 8 6]
 [1 6 3]]
mat:
 [[ 30  57  20]
 [ 57 149  73]
 [ 20  73  46]]
np.linalg.inv(mat):
 [[ 0.48628827 -0.37053571  0.37659439]
 [-0.37053571  0.3125     -0.33482143]
 [ 0.37659439 -0.33482143  0.38934949]]
```

7.5 ndarray 屬性

- dtype：ndarray 每個元素的資料型別 (可於創建矩陣時同時指定)
  - int, float, bool, D(複數)
- ndim：ndarray 的維度
- shape：ndarray 的大小，即矩陣 m*n (m 列 n 行)
- size：ndarray 的元素個數
- itemsize：ndarray 中每個元素記憶體大小 (單位 byte)

```python
import numpy as np

np1 = np.arange(12, dtype='D').reshape(3, 2, 2)
print(np1)
print('dtype=', np1.dtype)
print('shape=', np1.shape)
print('size=', np1.size)
print('itemsize=', np1.itemsize)
print('nbytes=', np1.nbytes)
```

```
[[[ 0.+0.j  1.+0.j]
  [ 2.+0.j  3.+0.j]]

 [[ 4.+0.j  5.+0.j]
```

```
  [ 6.+0.j  7.+0.j]]

 [[ 8.+0.j  9.+0.j]
  [10.+0.j 11.+0.j]]]
dtype= complex128
shape= (3, 2, 2)
size= 12
itemsize= 16
nbytes= 16.0
```

- nbytes：ndarray 的記憶體大小 (單位 byte)
  - real 和 imag：複數的實部和虛部
  - real 回傳複數的實部
- imag 回傳複數的虛部
- T：矩陣轉置

7.5.1 練習

- 使用 random 函式產生 3*5 的矩陣 A
- 使用 randint 函式產生 5*3 的矩陣 B
- 使用 arange 和 reshape 函式產生 3*5 的矩陣 C
- 使用 list 自建 5*3 的矩陣 D
- 計算 10*A + C 和 B - D
- 計算 B • C 和 C • D
- 比較 (B • C)T 和 CT • BT 是否相同?
- 求矩陣 E 的反矩陣

7.6 矩陣形狀轉換

- reshape(m, n, ⋯)：不改變原矩陣形狀
- resize(m, n, ⋯)：和 reshape 類似，但會改變原矩陣形狀
- ravel( ) / flatten( )：將多維矩陣轉換成一維矩陣，原矩陣不變
- shape = (m, n, ⋯)：用 tuple 來指定矩陣大小，會改變原矩陣

8 Matplotlib

9 網路爬蟲

```

import urllib.request as req
url = "https://huodalife.pixnet.net/blog/"
# 幫request加上一個header
request = req.Request(url, headers = {
    "User-Agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:72.0) Gecko
       /20100101 Firefox/72.0"
})

with req.urlopen(request) as response:
    data = response.read().decode("utf-8")

import bs4
root = bs4.BeautifulSoup(data,"html.parser")

titles = root.find_all("li", class_="title")
pubs = root.find_all("li", class_="publish")
for pub, title in zip(pubs, titles):
    print(pub.span.string,title.a.string)
```

10 Python 學習資源

- Python 3 教程
- Python 教程

11 0802 CNN

`https://reurl.cc/Y2gKo`: 各種常用的 library, function

12 以 python 下載網路資料

- import urllib.request as req

13 網路爬蟲

13.1 parse HTML

- what is HTML
- 精神：將程式模仿成一般 user

13.1.1 所需套件

- BeautifulSoup

pip install beautifulsoup4

13.2 範例：以程式抓取 PTT 電影板 header

13.2.1 原則

1. 要抓網路資料，就要先仔細觀察網頁內容

2. 要儘量欺騙網站自己是 browser

13.2.2 實作：取得 HTML 資料

1. 第一版

```
# 抓取PTT電影版header
import urllib.request as req
url = "https://www.ptt.cc/bbs/Stock/index.html"

with req.urlopen(url) as response:
    data = response.read().decode("utf-8")
print(data)
```

```
urllib.error.HTTPError: HTTP Error 403: Forbidden
```

被拒絕原因：這隻程式的行為不像一般使用者，被網站伺服器拒絕。403 Forbidden 是 HTTP 協議中的一個 HTTP 狀態碼（Status Code）。可以簡單的理解為沒有權限訪問此站，服務器收到請求但拒絕提供服務[3]。

2. 第二版: 加入 header

```
import urllib.request as req
url = "https://www.ptt.cc/bbs/Stock/index.html"
# 幫request加上一個header
request = req.Request(url, headers = {
    "User-Agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:72.0)
        Gecko/20100101 Firefox/72.0"
})

with req.urlopen(request) as response:
    data = response.read().decode("utf-8")
print('所取得的資料型態：',type(data))
print(data[700:1000])
```

```
所取得的資料型態： <class 'str'>
er">
        <div id="topbar" class="bbs-content">
                <a id="logo" href="/bbs/"> 批踢踢實業坊</a>
                <span>&rsaquo;</span>
                <a class="board" href="/bbs/Stock/index.html"><span class="board-label">
                <a class="right small" href="/about.html"> 關於我們</a>
                <a class="right small" href="/contact.html">
```

13.2.3 實作：解析 HTML 資料內容

1. 安裝解析 HTML 所需套件

```
pip install beautifulsoup4
```

2. 第三版: 解析 HTML

[3]維基百科: HTTP 403

```python
import urllib.request as req
url = "https://www.ptt.cc/bbs/Stock/index.html"
# 幫request加上一個header
request = req.Request(url, headers = {
    "User-Agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:72.0)
       Gecko/20100101 Firefox/72.0"
})

with req.urlopen(request) as response:
    data = response.read().decode("utf-8")

import bs4
root = bs4.BeautifulSoup(data,"html.parser")
print(root.title)
titles = root.find("div", class_="title")
print(titles)
print(titles.a.string)
titles = root.find_all("div", class_="title")
print(titles)
for title in titles:
    if title.a != None:
        print(title.a.string)
```

```
<title> 看板 Stock 文章列表 - 批踢踢實業坊</title>
<div class="title">
<a href="/bbs/Stock/M.1581985895.A.742.html">Re: [新聞] 蝗蟲抵達邊境 中國人自信：威脅不大</
</div>
Re: [新聞] 蝗蟲抵達邊境 中國人自信：威脅不大
[<div class="title">
<a href="/bbs/Stock/M.1581985895.A.742.html">Re: [新聞] 蝗蟲抵達邊境 中國人自信：威脅不大</
</div>, <div class="title">
<a href="/bbs/Stock/M.1581986080.A.0B9.html">[新聞] 武漢肺炎衝擊 蘋果率先示警營收無法達標</
</div>, <div class="title">
<a href="/bbs/Stock/M.1581987000.A.EE7.html">[新聞] 超預期 國巨 MLCC 漲價五成</a>
</div>, <div class="title">
<a href="/bbs/Stock/M.1581987018.A.192.html">[新聞] 武漢肺炎》上海美國商會：1/3 企業若工廠<
</div>, <div class="title">
<a href="/bbs/Stock/M.1581987846.A.95F.html">[新聞] 中芯國際 14nm 第二代 FinFET 批量須等到
</div>, <div class="title">
<a href="/bbs/Stock/M.1581989947.A.392.html">Re: [請益] 有人買股票而完全捨棄定存嗎?</a>
</div>, <div class="title">
<a href="/bbs/Stock/M.1581990753.A.EB3.html">[新聞] 斷絕華為貨源！美擬祭禁令限制中企取得</a
</div>, <div class="title">
<a href="/bbs/Stock/M.1581994926.A.A50.html">[標的] 6531 愛普</a>
</div>, <div class="title">
<a href="/bbs/Stock/M.1581996309.A.C55.html">[請益] 是不是該立法更限制外資匯出資金？ </a>
</div>, <div class="title">
<a href="/bbs/Stock/M.1581999672.A.14E.html">Re: [請益] 存股標的選擇</a>
</div>, <div class="title">
<a href="/bbs/Stock/M.1582000361.A.6A2.html">[標的] 1201 味全 多</a>
</div>, <div class="title">
<a href="/bbs/Stock/M.1582000701.A.2AB.html">Re: [新聞] 蝗蟲抵達邊境 中國人自信：威脅不大</
</div>, <div class="title">
<a href="/bbs/Stock/M.1582001420.A.3B8.html">[新聞] 用電大戶買綠電 電證分離更活絡</a>
</div>, <div class="title">
```

```
<a href="/bbs/Stock/M.1422199105.A.84E.html">[公告] 精華區導覽 Q&amp;A</a>
</div>, <div class="title">
<a href="/bbs/Stock/M.1541252211.A.8BE.html">[公告] Stock 板規 V2.6             (2019/06/
</div>, <div class="title">
<a href="/bbs/Stock/M.1579935755.A.395.html">[公告] 關於武漢肺炎新聞規範</a>
</div>, <div class="title">
<a href="/bbs/Stock/M.1581919202.A.CF2.html">[閒聊] 2020/02/17 盤後閒聊</a>
</div>, <div class="title">
<a href="/bbs/Stock/M.1581985803.A.F3E.html">[閒聊] 2020/02/18 盤中閒聊</a>
</div>]
Re: [新聞] 蝗蟲抵達邊境 中國人自信：威脅不大
[新聞] 武漢肺炎衝擊 蘋果率先示警營收無法達標
[新聞] 超預期 國巨 MLCC 漲價五成
[新聞] 武漢肺炎》上海美國商會：1/3 企業若工廠
[新聞] 中芯國際 14nm 第二代 FinFET 批量須等到 2020
Re: [請益] 有人買股票而完全捨棄定存嗎?
[新聞] 斷絕華為貨源！美擬祭禁令限制中企取得
[標的] 6531 愛普
[請益] 是不是該立法更限制外資匯出資金?
Re: [請益] 存股標的選擇
[標的] 1201 味全 多
Re: [新聞] 蝗蟲抵達邊境 中國人自信：威脅不大
[新聞] 用電大戶買綠電 電證分離更活絡
[公告] 精華區導覽 Q&A
[公告] Stock 板規 V2.6             (2019/06/20)
[公告] 關於武漢肺炎新聞規範
[閒聊] 2020/02/17 盤後閒聊
[閒聊] 2020/02/18 盤中閒聊
```

13.3 參考資料

- https://www.youtube.com/watch?v=9Z9xKWfNo7k

14 如何把 python 程式 publish 成網站應用程式